Which apartments are over-priced? Ensembles, market segments and a neural network on krisha.kz ads

Endterm: the Week 5–8 methods applied to our 4,815 Astana and Almaty listings, on the same saved test split as the Midterm

Course project, Endterm (Week 9) · Machine Learning Algorithms · Direction 1 (Apartments for sale) · Regression

Rakhat Balgabekov, Aitbek Kaiyrbekuly · Group SE-2424 · November 2026

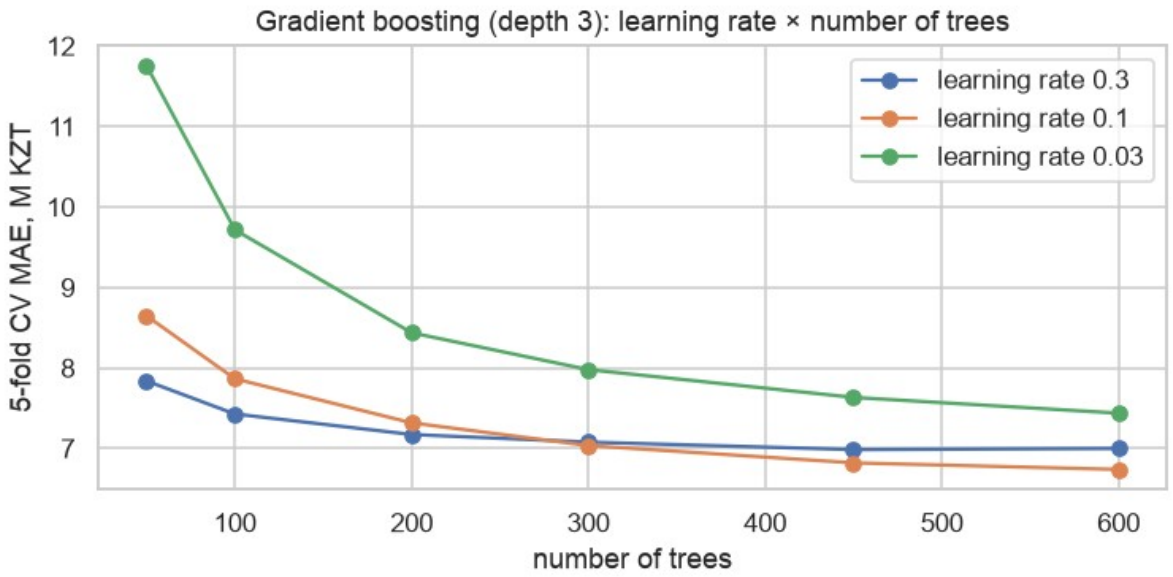
Question, data and what changed since the Midterm

Which apartments in Astana and Almaty are asked above or below what comparable apartments go for, and by how much?

- **Same data** 4,815 krisha.kz ads scraped 4–8 Oct 2026; target = asking price, KZT
- **Same split** 963 test ads saved at the Midterm, read from the file, used once (Section 6)
- **Same metric** MAE in KZT (the user's money) and MdAPE (registration goal: median error < 10 %)
- **Midterm result** linear model on log price, test MAE 8.65 M, MdAPE 11.3 %, goal not reached
- **Users** buyer flags an ad > 20 % above the estimate; seller gets a starting price; realtor a reference

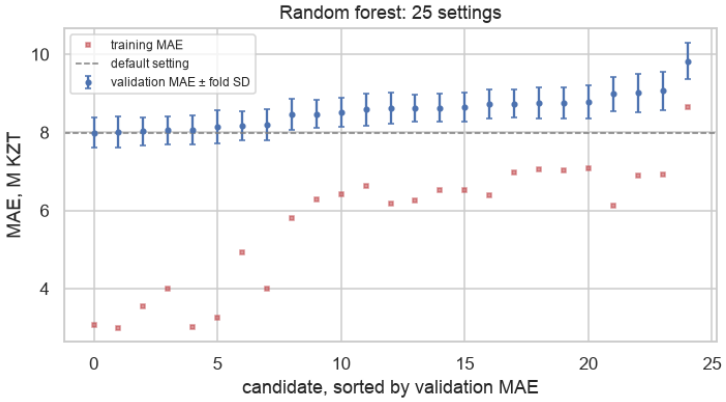
- **Fix 1 · question** from “predict the price” to “which ads deviate from the market, by how much”
- **Fix 2 · data ethics** free text checked for contact data: 0 phone numbers, 7 realtor handles masked
- **Fix 3 · features** has_detail dropped (a scraper artefact); 10 ads with coordinates in the wrong city → missing
- **Fix 4 · model choice** by 5-fold CV MAE only; the Midterm had picked by test MAE
- **Added** forest, boosting, XGBoost, LightGBM, randomised search, blend, K-Means, DBSCAN, PCA, segment feature, PyTorch MLP

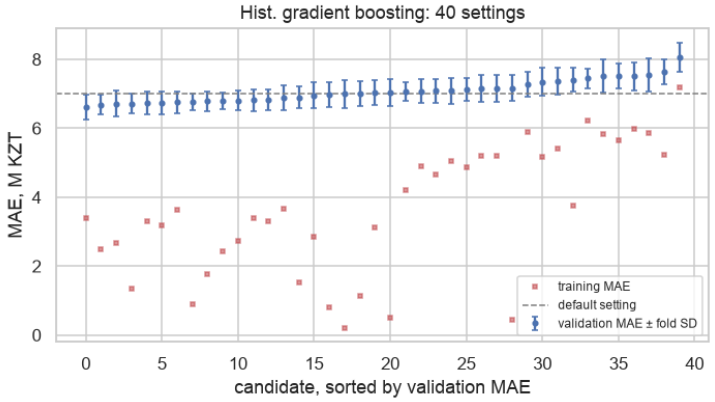
Ensembles: the forest removes the tree's variance, boosting its bias

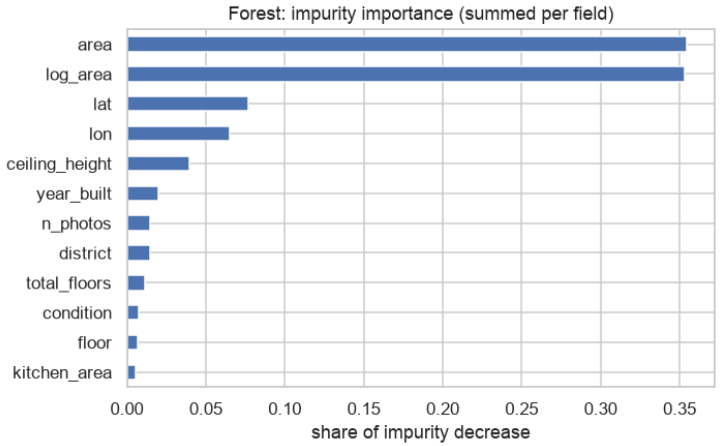


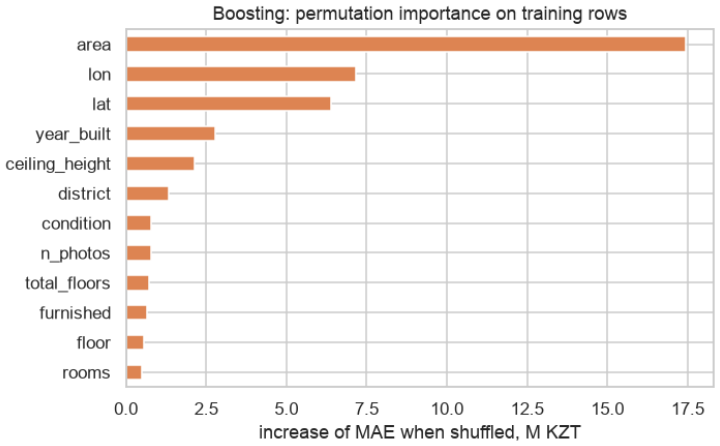
- **Fully grown tree** train MAE 0.00 → CV 11.25 ± 0.73 M: memorises, high variance
- **Random forest, 300 trees** CV 8.00 ± 0.37 M, SD halved: averaging cancels the variance
- **Gradient boosting, 100 × depth 3** CV 7.87 ± 0.33 M, smallest train→val gap (1.3 M): shallow trees, bias removed by summing
- **Histogram boosting (default)** CV 7.01 ± 0.38 M, 2.1 M below the linear model (9.10 ± 0.61) = 3–4 fold SDs
- **Learning rate × trees** 0.3 flattens at 450 trees (6.99 M), 0.1 still improving at 600 (6.74 M), 0.03 underfits (7.44 M); differences ≈ one fold SD

Tuning bought one fold SD for boosting and nothing for the forest; XGBoost and LightGBM confirm the score



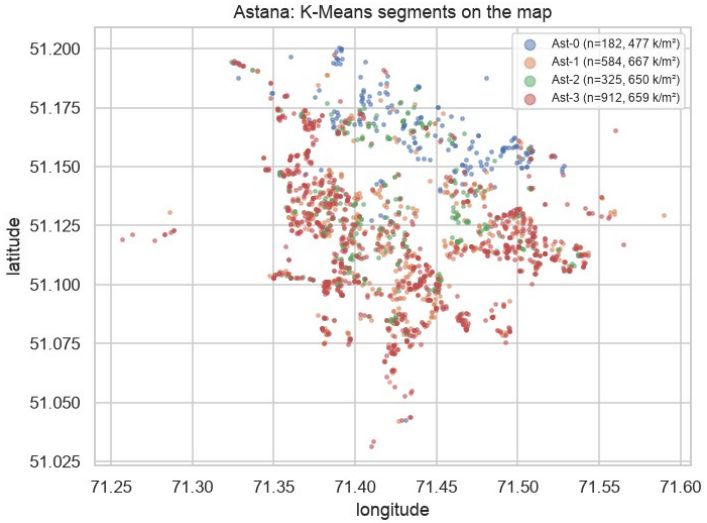


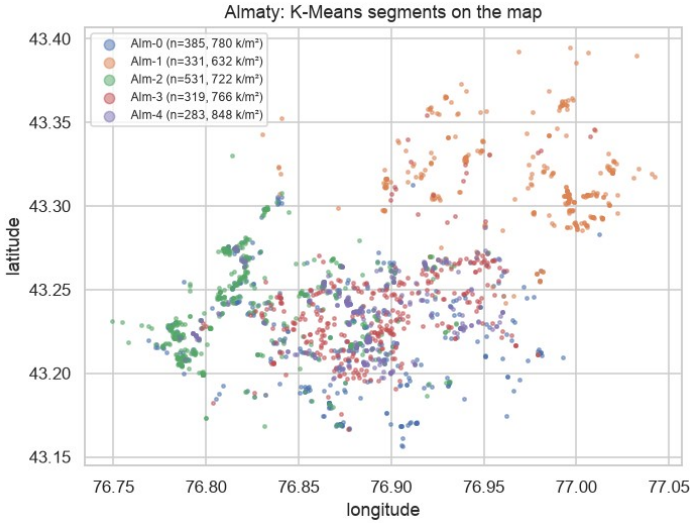


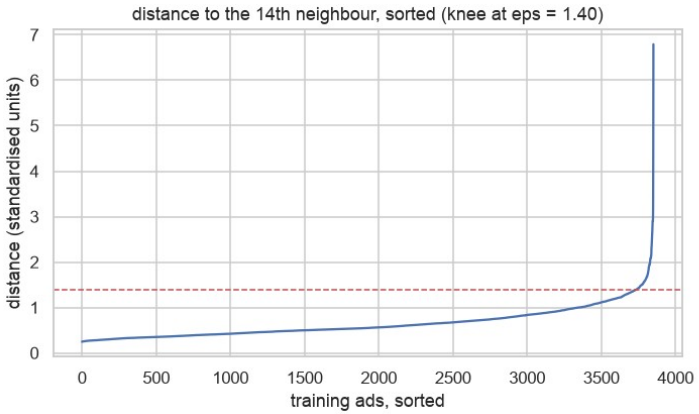


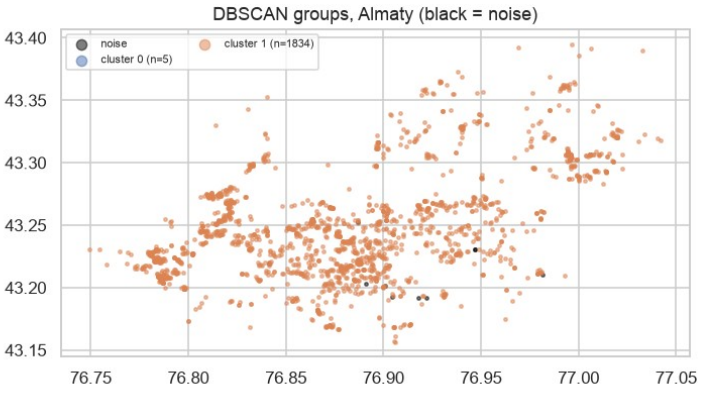
- **RandomizedSearchCV** 25 forest + 40 boosting + 20 XGBoost + 20 LightGBM settings, same 5 folds, scored by MAE in KZT
- **Forest** best 7.99 ± 0.38 M vs default 8.00: insensitive to its settings
- **Boosting** best 6.62 ± 0.35 M vs default 7.01: lr 0.12, 7 leaves, 600 iterations, 5 rows/leaf; the five best within 0.11 M, all with many iterations and small leaves
- **XGBoost 6.64 ± 0.32, LightGBM 6.75 ± 0.28** three libraries within a third of a fold SD; the 6.6 M floor belongs to the data, not the library
- **OOB check** forest OOB MAE 7.66 M vs CV 7.99 M, the two estimates agree
- **Permutation importance** shuffling area +17.5 M MAE, lon +7.2, lat +6.4, year +2.8, ceiling +2.1, district only +1.3: coordinates beat district names 5×
- **Blend forest + boosting** 6.95 M, between its parents; extra trees 8.69 M: no help

Market segments: K-Means within each city; DBSCAN finds one dense cloud per city plus 29 unusual ads



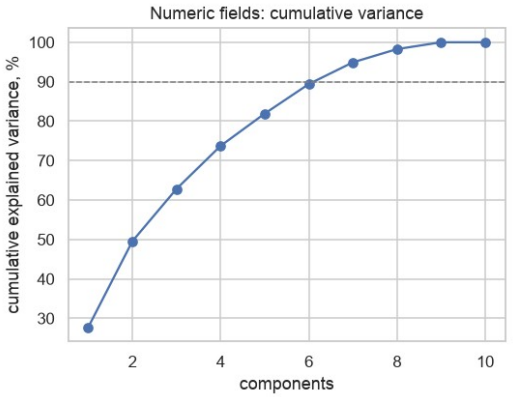


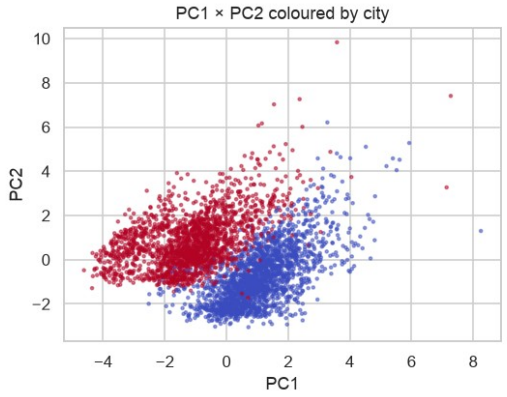


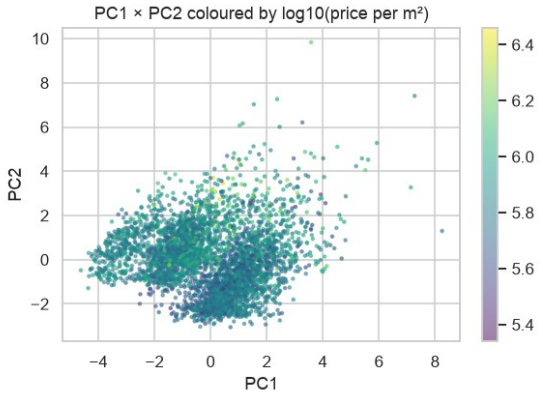


- **Inputs** log area, rooms, floor, floors in building, year, lat, lon; price never an input
- **k chosen per city** silhouette + inertia + stability (ARI on 80 % subsamples ≥ 0.9, no cluster < 30 ads) → Astana 4 segments (silhouette 0.21, ARI 0.97), Almaty 5 (0.25, 0.97)
- **Segments in words** Astana: mass market 45 m² new (912 ads), large family flats 92 m², tall-tower flats floor 13/18, old low-rise 2006 at 477 k/m² (cheapest). Almaty: premium high-rise 848 k/m², club-house flats 87 m², Soviet centre 1976 at 766 k/m², small new west, cheap north-east 632 k/m²
- **Silhouette 0.21–0.25** cuts through a continuum, a description, not islands
- **DBSCAN, eps 1.40 at the knee, min_samples 14** 2 clusters = the two cities, 29 noise ads (0.8 %)
- **Noise = real unusual objects** 140 m² on floor 42/43, a 187 m² 6-room penthouse (listed twice), a 1-room flat on 11/34; kept, but used as a “few comparables” warning

PCA describes the table but does not help the model; a segment feature adds nothing

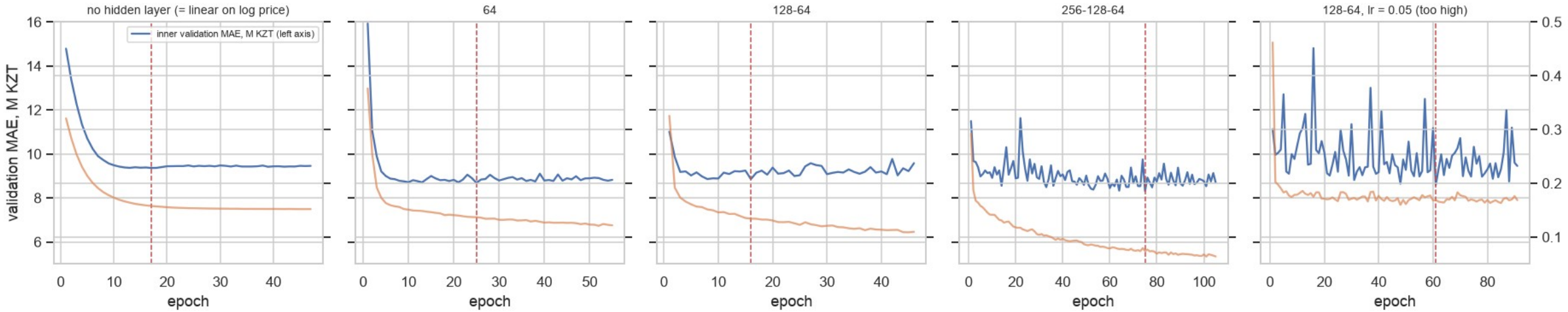






- **10 numeric fields** 5 components for 80 %, 7 for 90 % of the variance; the last one is 0 % (lat and lon anti-correlated across the two cities)
- **PC1 = new high-rise in Astana vs old low-rise in Almaty** total floors 0.41, lat 0.40, year 0.34, lon −0.40
- **PC2 = size** area 0.50, rooms 0.52, photos 0.35
- **KNN on components** 11.5 M (90 % var.) vs 11.1 M raw; Ridge 10.6 M vs 9.1 M raw: every reduction costs
- **Midterm KNN on size + location only (10.2 M) beats both** columns chosen by meaning beat directions chosen by variance
- **Boosting + in-fold K-Means segment** 6.65 ± 0.31 vs 6.62 ± 0.35 M: the model already carves those regions itself

A PyTorch network ties the linear model and loses to boosting by six fold SDs



- **Setup** same encoded inputs (71 columns), standardised log price, L1 loss, Adam, dropout, weight decay, early stopping on 15 % of the fold, same 5 folds
- **No hidden layer = linear by gradient descent** 9.26 ± 0.57 M vs 9.10 for the Midterm linear model, the loop is right
- **64 / 128-64 / 256-128-64 units** 8.82 / 8.90 / 8.86 M, all within 0.1 M; scikit-learn MLP 8.86 M confirms
- **Curves** training loss keeps falling, validation stops after 10–30 epochs and wanders: memorising; lr 0.05 makes it jump between 8.5 and 10.5 M
- **Verdict** 2.2 M behind tuned boosting (6.62 M) on every fold; 3,852 tabular rows with threshold effects are tree territory, 0.3–0.7 s per fit on one CPU thread

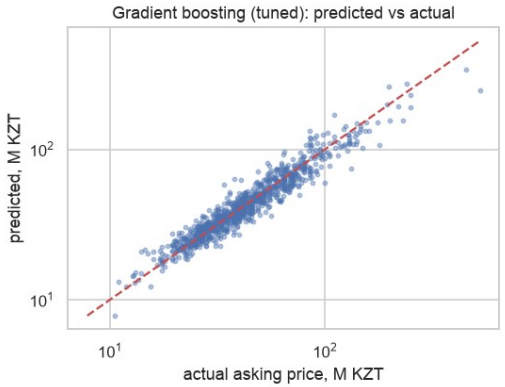
One table, chosen by CV, tested once: tuned boosting reaches MdAPE 9.1 %, the 10 % goal is met

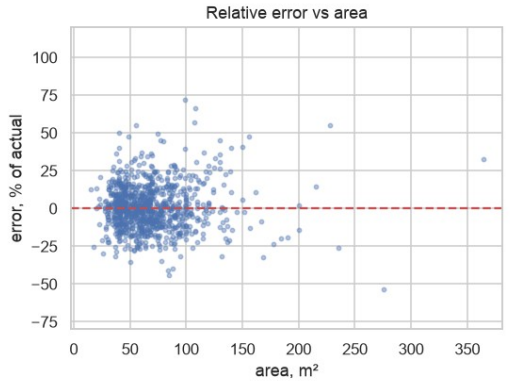
model	CV MAE (M KZT)	CV MdAPE %	Test MAE (M KZT)	Test MdAPE %	Test R^2	within ±20 %
Gradient boosting (tuned) ⋆	**6.62 ± 0.35**	**8.9**	**6.291**	**9.13**	**0.855**	**84%**
XGBoost (tuned)	6.64 ± 0.32	8.8	6.269	8.94	0.862	85%
LightGBM (tuned)	6.75 ± 0.28	9.0	6.193	9.0	0.877	84%
Blend forest + boosting	6.95 ± 0.33	9.0	6.462	8.68	0.854	84%
Hist. boosting (default)	7.01 ± 0.38	9.2	6.624	9.06	0.849	85%
Random forest (tuned)	7.99 ± 0.38	10.2	7.222	9.98	0.834	80%
PyTorch MLP, 64 units	8.82 ± 0.60	11.4	8.327	10.99	0.735	75%
sklearn MLP 128-64	8.86 ± 0.53	12.2	8.266	11.85	0.803	74%
Midterm linear (log price)	9.10 ± 0.61	12.1	8.66	11.27	0.712	73%
Midterm tree (depth 9)	10.19 ± 0.50	13.3	9.421	13.46	0.751	67%
Realtor rule (district price/m^2)	10.45 ± 0.39	14.6	9.945	14.35	0.691	62%
Dummy (median)	20.81 ± 0.75	32.1	20.402	33.06	-0.067	32%

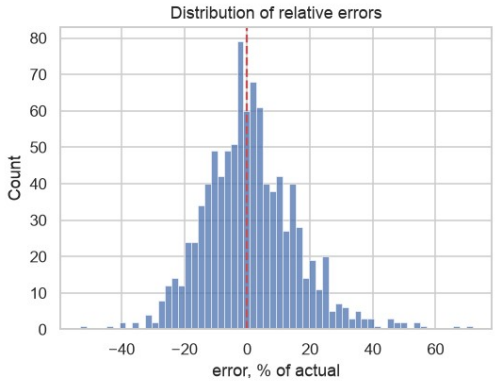
- **Chosen by CV MAE** tuned histogram boosting, 6.62 ± 0.35 M
- **Test, used once** MAE 6.29 M (bootstrap 5.56–7.14), MdAPE 9.13 % (8.31–9.88), R^2 0.855
- **vs Midterm on the same 963 ads** 8.66 M, 11.27 %, R^2 0.712: error in money −27 %
- **Within ±10 % / ±20 %** 54 % / 84 % of test ads (Midterm 45 % / 73 %)
- **LightGBM, XGBoost, blend and default boosting look as good on the test column (6.2–6.6 M)** we do not switch, that would be choosing on the test set; three libraries agreeing is what makes the number trustworthy

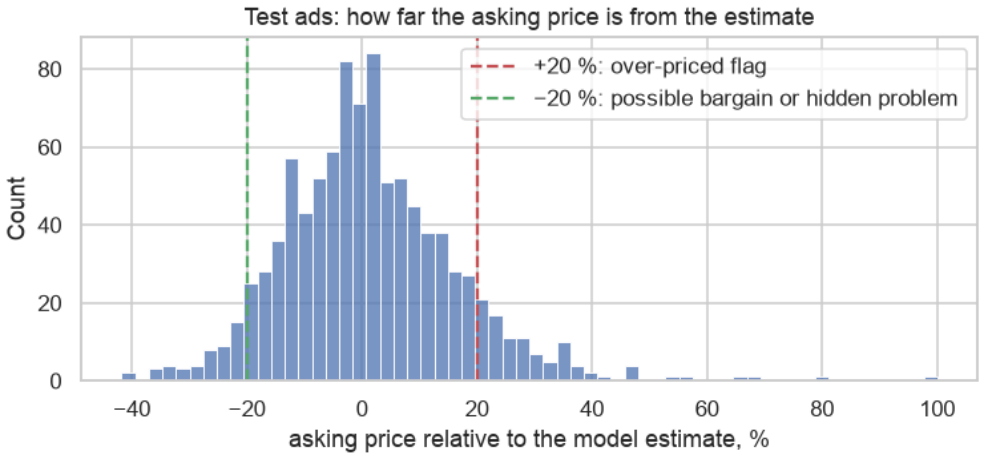
CV = 5-fold cross-validation on the 3,852 training rows (mean ± fold SD). Test = the 963 ads saved at the Midterm, used once. Model chosen by CV MAE.

Errors: the expensive bands improved most; what is left sits in the text









- **By price band (MdAPE, Endterm vs Midterm)** 20–50 M 8 % vs 10 %; 80–150 M 12.9 % vs 21.9 %; > 150 M 16.6 % vs 31.8 %
- **Bias** cheap ads over-estimated by 9 %, luxury under-estimated by 14 % (15 ads > 150 M)
- **Worst cases** “cash only, assignment of rights” (13.9 M predicted 20.8), rough state, a 276 m² designer duplex at 525 M predicted 245: text-only reasons or no comparables
- **Midterm's worst 8** 364 m² penthouse 133 % → 33 %, Nura flat 68 % → 3 %; four of eight stay above 40 %
- **Buyer's flag** 93 of 963 ads (10 %) ask > 20 % above the estimate; owners and agents equally (10 %); 28 ads with no comparables get a warning (MdAPE 12.9 % vs 9.0 %)

Conclusions, limitations and the plan for the Final

- **Answer** structured fields give a 9.1 % median error; one ad in ten asks > 20 % above comparable apartments, one in twenty > 20 % below
- **Ensembles** −27 % error, mostly in the expensive bands; tuning worth one SD; XGBoost and LightGBM land at the same score; blend and extra trees no help
- **Unsupervised** one dense cloud per city; segments describe, DBSCAN flags unusual ads; PCA shows 7 real dimensions, none of them price
- **Network** ties linear, loses to boosting; not worth it on 3,852 tabular rows
- **Importance** size and coordinates carry the model; what remains is not in the fields
- **Limitations** asking ≠ sale price; one October snapshot, one site; 15 luxury test ads; the test set is seen at every stage; tuning on the comparison folds
- **Final 1 · text** TF-IDF of the sanitised description on cross-fitted residuals of the boosting model, judged on the worst cases
- **Final 2 · photos** pretrained-CNN embeddings of up to 3 photos per ad as a second residual correction
- **Final 3 · app** fields + text + photos → estimate, segment-based interval, over-pricing flag with the “few comparables” warning
- **Final 4 · honest check** re-scrape in November as a temporal hold-out; the saved split stays unchanged